Documentation technique

Site Web de
la Boulangerie du Village

Lucila VANHALLE (DSP09 - TC3)

PHP - Bases de données

Sommaire

Documentation technique **0**
Sommaire **1**
1. Architecture & sécurité **2**
1.1. Choix technologiques 2
1.2. Architecture du site 2
a) Fichiers appelés plusieurs fois 2
b) Pages côté client 3
c) Pages côté admin 6
d) Fichiers pour les fonctionnalités 8
1.3. Sécurité du site 14
a) Hachage et authentification 14
b) Sécurisation de la base de données 14
c) Sanitisation côté affichage 15
d) Vérification de la méthode de réception des données 15
e) Sécuriser l'accès des fonctionnalités admin 15
2. Modélisation de la base de données **16**
2.1. Modèle Conceptuel des Données (MCD) 16
2.2. Modèle Logique de Données (MLD) 16
2.3. Présentation des tables 16
2.4. Étapes de la création de la base de données MySQL 17
3. Guide d'Installation **20**
3.1. Pré-requis (Installation d'un serveur local) 20
3.2. Importation du dump de la base de données (.sql) 20
3.3. Configuration de la connexion à la base de données 20
3.4. Déploiement et lancement du site 21

1. Architecture & sécurité

1.1. Choix technologiques

HTML/CSS
PHP
MySQL
Docker

1.2. Architecture du site

J'ai développé 21 fichiers répartis en bloc selon leur utilisation.

a) Fichiers appelés plusieurs fois

dbconfig.php :

Il s'agit du fichier qui permet de se connecter à ma base de données MySQL depuis PHP. J'ai choisi de faire un fichier séparé au lieu de tout réécrire dans chaque fichier afin de gagner du temps car il suffit d'inclure le fichier avec l'instruction include dans chaque fichier quand j'en ai besoin.

header.php et footer.php :

En cours de CMS, nous avons appris à scinder la structure d'une page en plusieurs fichiers PHP donc j'ai décidé d'utiliser à nouveau cette façon de faire. En incluant ces fichiers (toujours avec l'instruction include) dans toutes mes pages, cela garantit la présence de ma barre de navigation (header) et de mon footer dans celles-ci. De plus, il suffit de modifier ces fichiers seuls pour les modifier sur toutes les pages.

Dans le fichier header.php, j'ai mis en place une condition if qui permet de vérifier qui est connecté sur le site grâce à l'attribut role défini dans la table utilisateur de la base de données. S'il s'agit d'un client (0), la barre nav reste telle qu'elle, si c'est l'admin (1), on affiche en plus dans la barre navigation un lien vers le dashboard du boulanger.

```php
<?php
// On vérifie si c'est l'admin grâce à son rôle
if (isset($_SESSION['utilisateur_id'])) {
    if (isset($_SESSION['role']) && $_SESSION['role'] == "1") { ?>
        <a href="admin.php">Mon Dashboard de boulanger</a>
    <?php } ?>
```

b) Pages côté client

index.php :

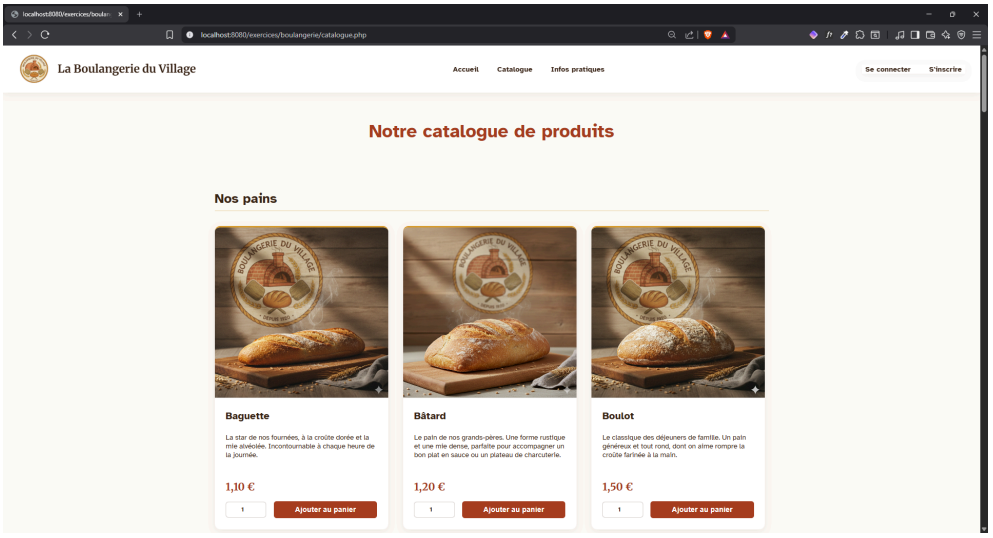
Il s'agit du fichier de la page d'accueil du site.

Elle y présente un message d'accueil dans une bannière et trois produits affichés grâce à une requête SQL.

```php
// On limite à 3 pour n'avoir que 3 produits affichés sur la page
$sql = "SELECT nom, image, prix FROM produit LIMIT 3";
$result = $conn->query($sql);
```

catalogue.php :

Un fichier permettant aux clients (mais aussi à l'admin s'il veut voir à quoi cela ressemble) de voir le catalogue des produits proposés par la boulangerie organisés par catégories (pains, viennoiseries ou pâtisseries).



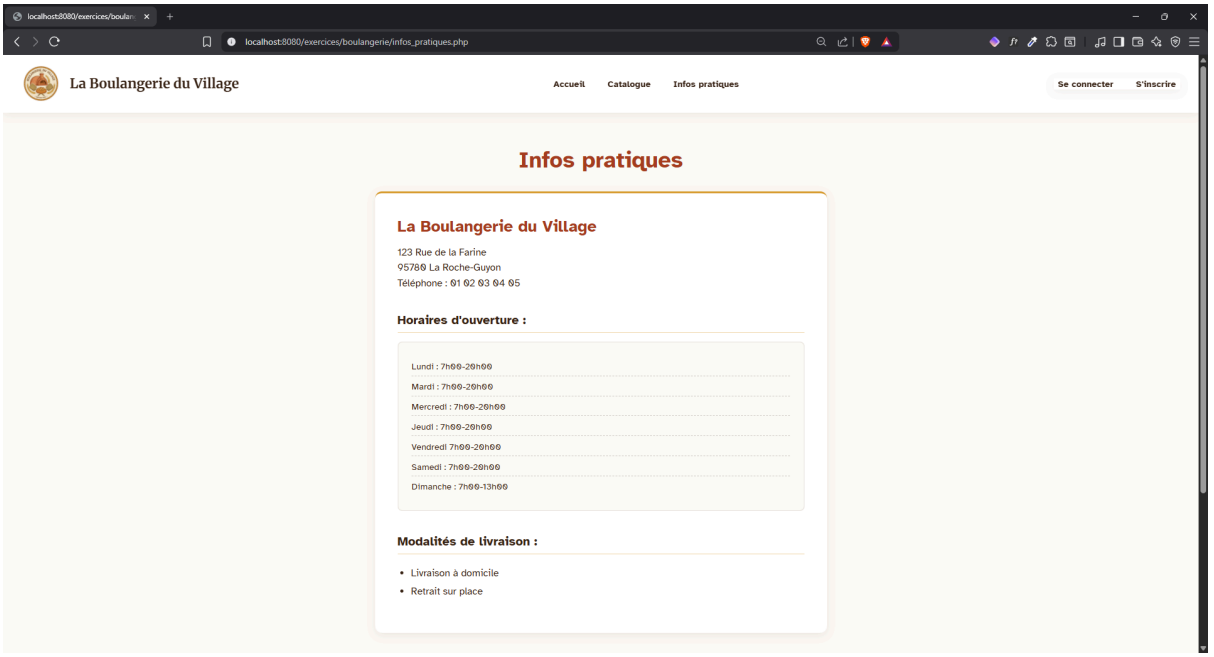
Pour cela, j'ai utilisé une requête contenant une jointure entre la table produit et la table categorie ainsi qu'une boucle while. Dans cette boucle, une variable mémorise la catégorie en cours de lecture afin de n'afficher l'en-tête (ex : Nos pains) que lorsque la catégorie change.

```php
<?php
// Requête simple pour grouper les produits par catégorie
$sql = "SELECT produit.*, categorie.nom AS nom_categorie
FROM produit
JOIN categorie ON produit.id_categorie = categorie.id
ORDER BY produit.id_categorie, produit.nom";
$result = $conn->query($sql);

$categorie_actuelle = "";

if ($result->num_rows > 0) {
    // Tant qu'il reste une ligne de produit à lire dans les résultats, on l'extrait sous forme de tableau associatif
    while ($row = $result->fetch_assoc()) {
        // Si la catégorie actuelle est différente de celle dans la ligne en cours, il s'agit donc d'un nouvel en-tête
        if ($categorie_actuelle != $row['nom_categorie']) {
            echo "<div class='categorie-en-tete'>";
            echo "<h2 class='categorie-titre'>Nos " . htmlspecialchars($row['nom_categorie']) . "</h2>";
            echo "</div>";

            $categorie_actuelle = $row['nom_categorie'];
        }
?>
```

Pour les produits, j'ai décidé de les représenter sous forme de cartes contenant une image, le nom, la description, le prix, un champ pour la quantité et un bouton "Ajouter dans le panier".

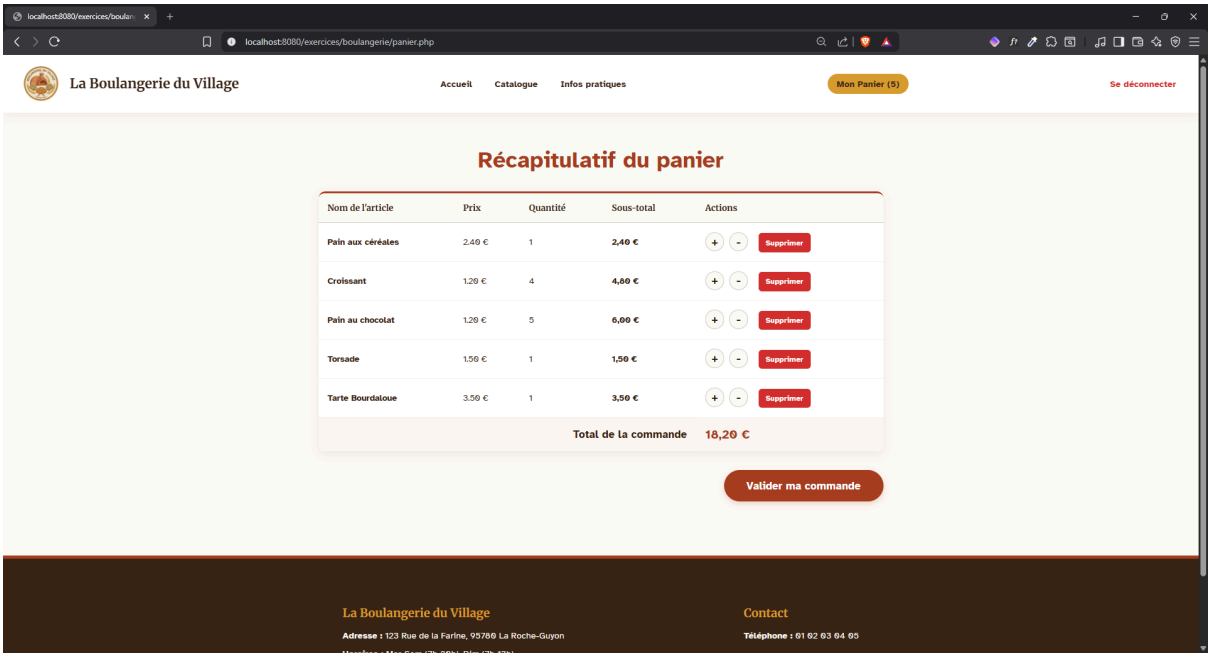
infos_pratiques.php :

Il s'agit d'une page statique qui a pour but de rassurer et informer les clients en affichant les horaires d'ouverture, l'adresse physique de la boulangerie et surtout les modalités de livraison.



panier.php :

Cette page est destinée au client et sert de récapitulatif des produits qu'il a ajouté avant la validation de la commande.



Les données du panier sont stockées dans la superglobale $_SESSION.

```
// On vérifie si le panier existe et n'est pas vide
if (isset($_SESSION["panier"]) && !empty($_SESSION["panier"])) {
```

Sur cette page, le client peut modifier la quantité d'un produit ou de le supprimer s'il change d'avis ou ajoute un produit par erreur grâce à des liens utilisant la méthode GET.

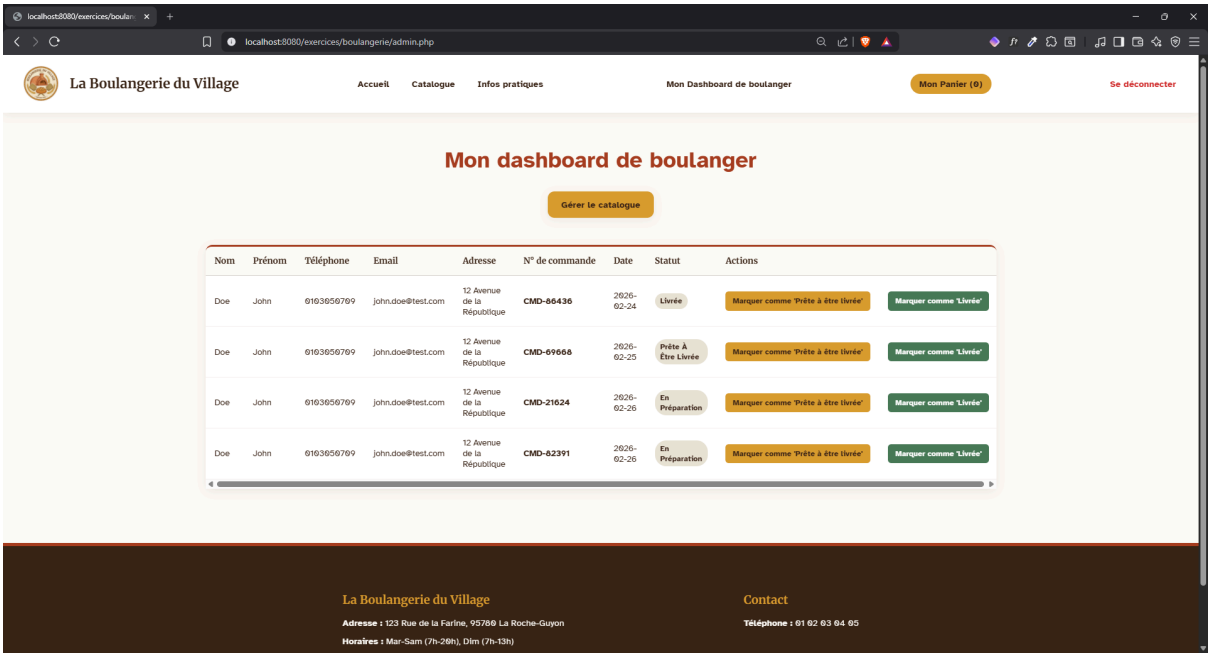
```
51        // On fait deux liens qui envoient par la méthode GET l'id du produit en question ainsi que l'action à effectuer
52        echo "<td><a class='btn-qty' href='modifier_quantite.php?id_produit=" . $id_produit . "&action=ajouter'>+</a>";
53        echo "<a class='btn-qty' href='modifier_quantite.php?id_produit=" . $id_produit . "&action=retirer'>-</a>";
54        echo "<a class='btn-action btn-supprimer' href='supp_panier.php?id_produit=" . $id_produit . "'>Supprimer</a></td>";
55        echo "</tr>";
56    }
```

c) Pages côté admin

Sur toutes ces pages admin, on inclut la vérification du role pour garantir l'accès seulement à l'admin.

admin.php :

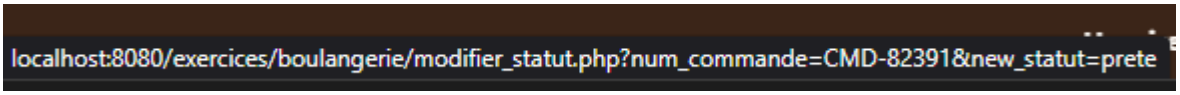
C'est la page du Dashboard du boulanger. Elle n'est accessible que si c'est l'admin qui est connecté donc c'est pour cela que session_start() et la condition vérifiant le role de l'utilisateur est primordial dans ce fichier.



Sur cette page, l'admin a une vision globale des commandes de ses clients dans un tableau. Les données sont récoltées et affichées grâce à une requête SQL et à la boucle while qui s'assure que tant qu'il y a de ligne dans le résultat de la requête, on l'affiche sous forme de ligne de tableau.

Ainsi, le tableau contient le nom et le prénom du client, son numéro de téléphone, son email et son adresse, le numéro de la commande, la date de cette dernière et son statut actuel.

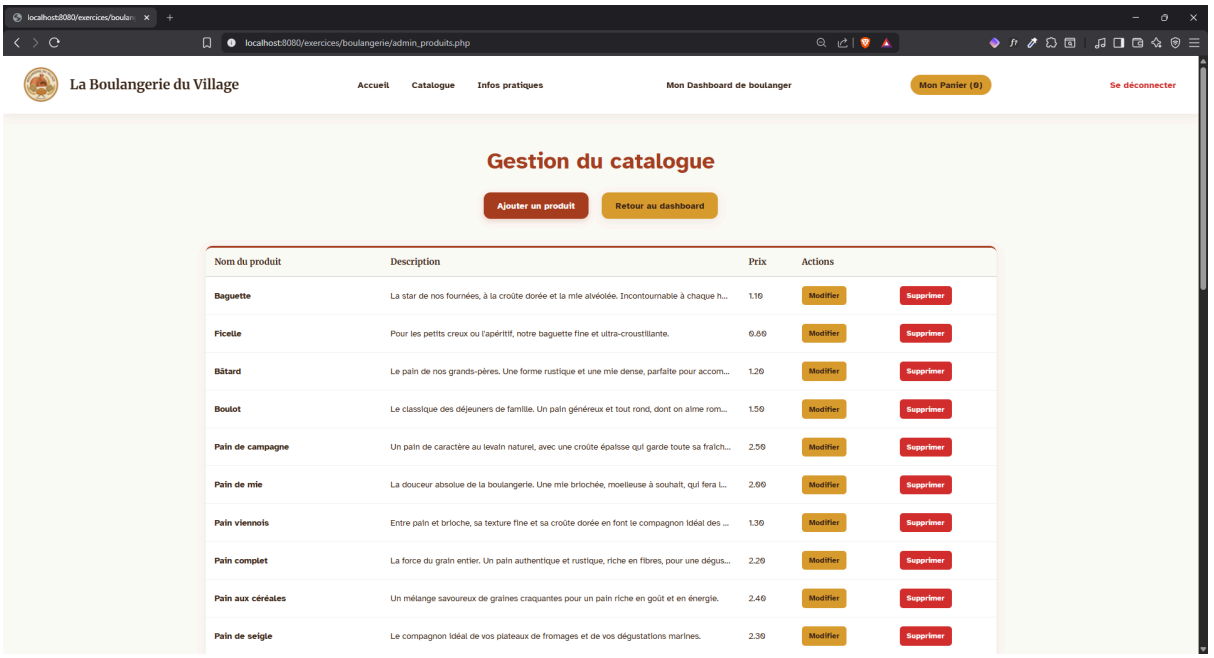
Pour chaque commande, l'admin peut changer le statut et le passer à "Prête à être livrée" ou "Livrée" grâce aux deux liens utilisant la méthode GET.



```
// Liens utilisant la méthode GET pour envoyer le numéro de commande et l'action à effectuer au fichier modifier_statut.php
echo "<td><a class='btn-action btn-prete' href='modifier_statut.php?num_commande=" . $row["num_commande"] . "&new_statut=prete'>Marquer comme 'Prête à être livrée'</a></td>";
echo "<td><a class='btn-action btn-livree' href='modifier_statut.php?num_commande=" . $row["num_commande"] . "&new_statut=livree'>Marquer comme 'Livrée'</a></td>";
```

admin_produits.php :

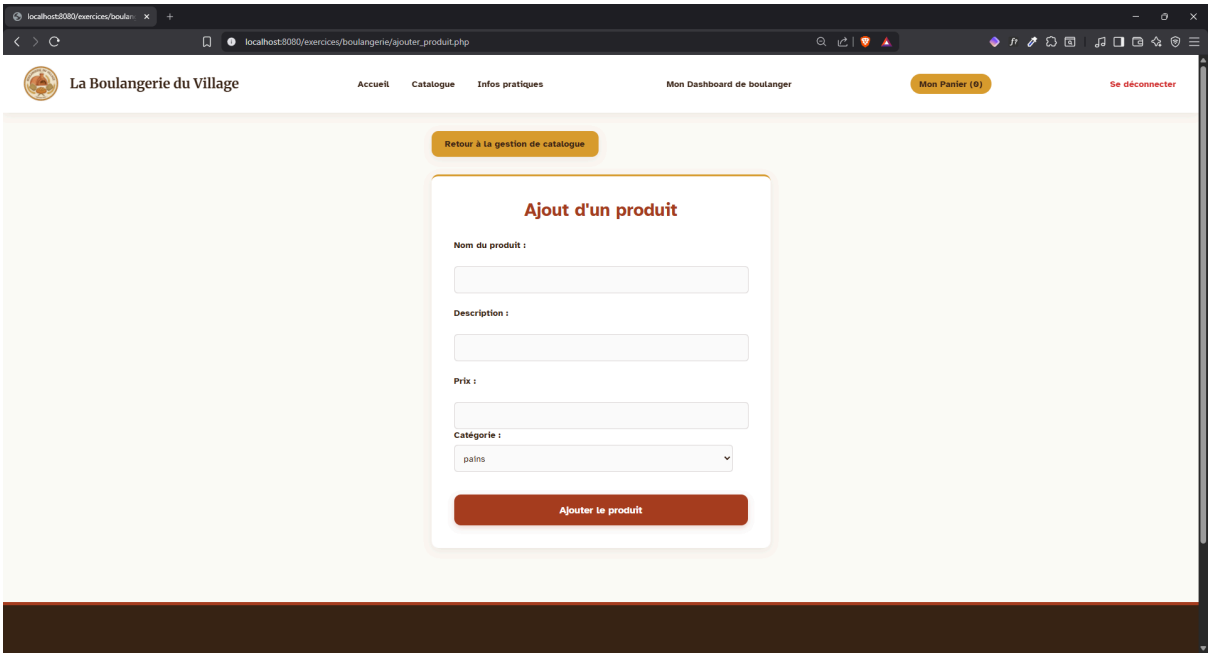
Cette page permet la gestion du catalogue.



Ici, l'admin peut modifier, supprimer ou ajouter un produit.

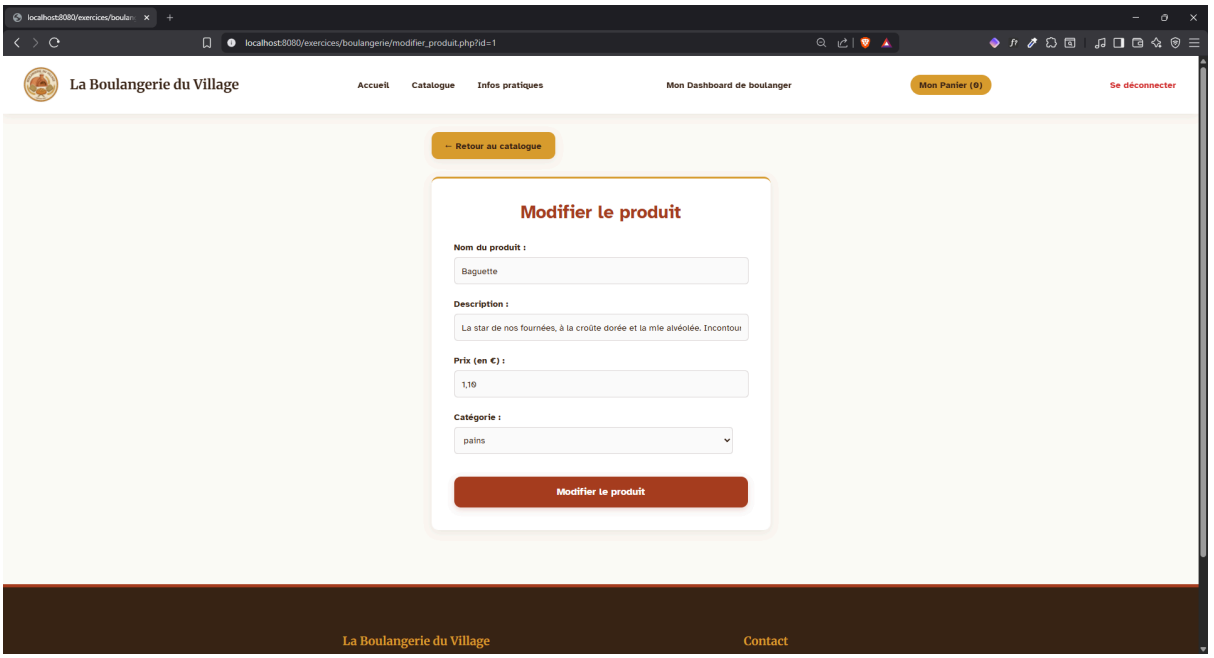
ajouter_produit.php :

Cette page est celle qui s'affiche si l'on clique sur le lien "Ajouter un produit" dans la gestion du catalogue.



modifier_produit.php :

Cette page, quant à elle, s'affiche lorsque l'admin clique sur le lien "Modifier" dans une des lignes du tableau gestion du catalogue.

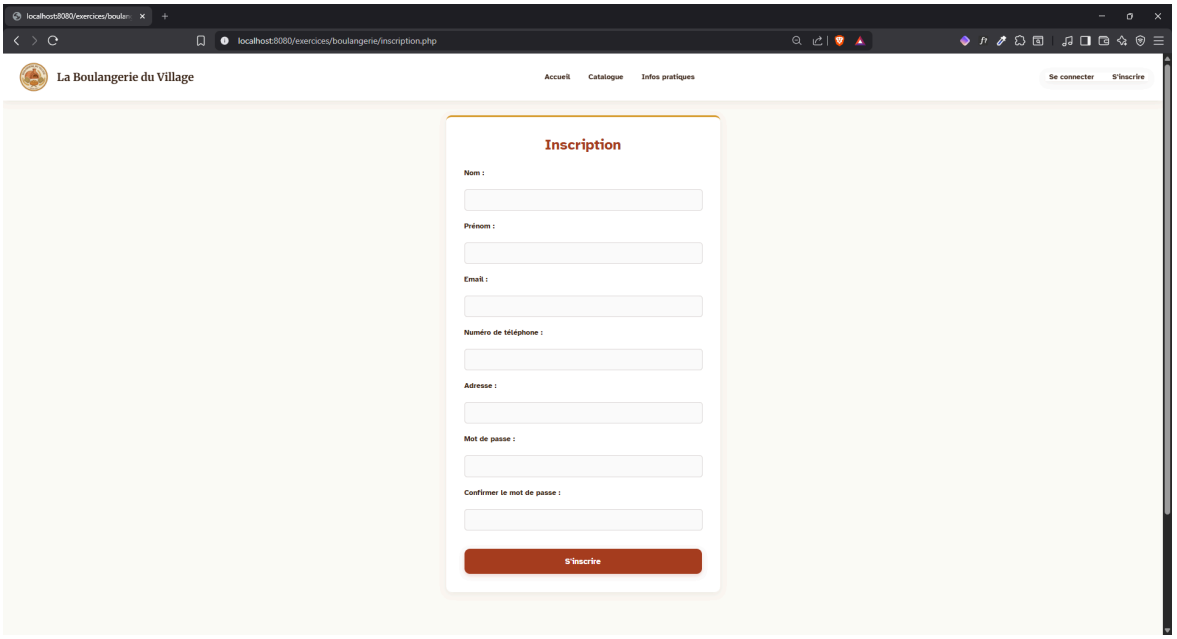


d) Fichiers pour les fonctionnalités

inscription.php :

Il s'agit d'une page qui affiche un formulaire d'inscription. Ce fichier reçoit les données du formulaire de manière invisible avec la méthode

POST. Il vérifie que les champs ne sont pas vides (avec !empty()) puis il enregistre le nouveau client dans la table utilisateur grâce à la requête préparée.

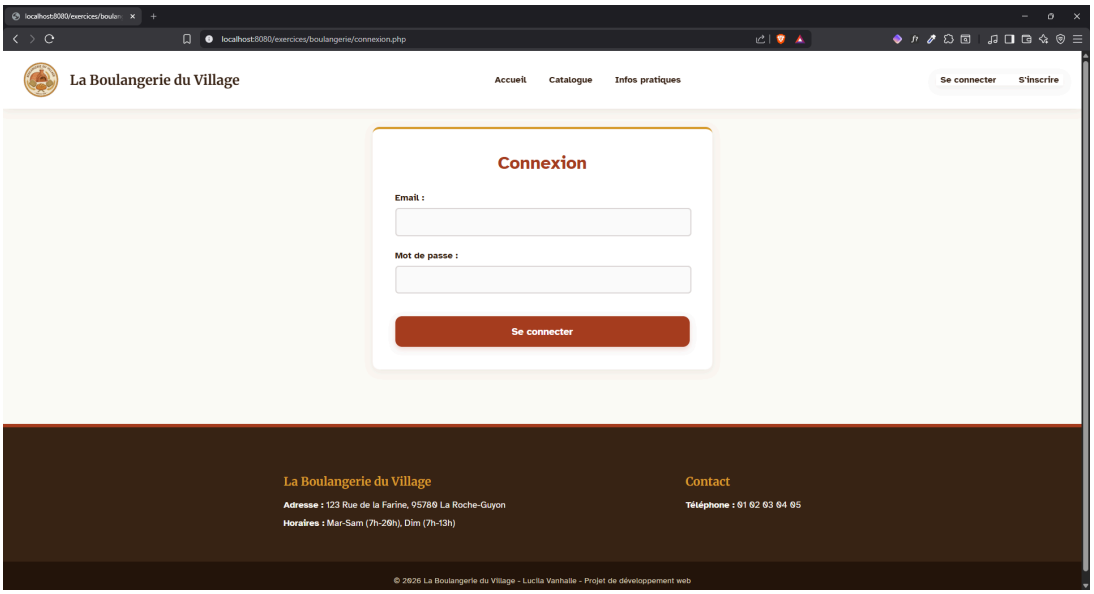


Dans ce formulaire, le mot de passe n'est jamais stocké directement en clair, il est d'abord haché grâce à la fonction password_hash().

```php
// On vérifie bien si les mots de passe sont identiques
if ($_POST['password'] !== $_POST['password_confirmation']) {
    echo "Les mots de passe ne correspondent pas.";
} else {
    $mot_de_passe = password_hash($_POST['password'], PASSWORD_DEFAULT);
```

connexion.php :

Il s'agit également d'une page qui affiche un formulaire de connexion.



Ce fichier reçoit l'email et le mot de passe. Il va interroger la base de données pour voir si l'email existe puis comparer le mot de passe saisi avec celui haché en base de données. Puisque ce mot de passe est haché, il faut utiliser la fonction password_verify(mot de passe saisi, mot de passe haché).
Si tout correspond, il ouvre une session pour le client en enregistrant son id, son prénom et son attribut role dans la superglobale $_SESSION.
Enfin, on redirige l'utilisateur vers l'accueil grâce à la fonction header() suivi de son exit().

```php
if ($utilisateur && password_verify($mdp_saisi, $utilisateur['mot_de_passe'])) {
    $_SESSION['utilisateur_id'] = $utilisateur['id'];
    $_SESSION['prenom'] = $utilisateur['prenom'];
    $_SESSION['role'] = $utilisateur['role'];

    header("Location: index.php");
    exit();
```

deconnexion.php :

Ce fichier programme le mécanisme de déconnexion de l'utilisateur. Il détruit la session en cours grâce à la fonction session_destroy() et renvoie l'utilisateur vers l'accueil.

```php
<?php
ini_set('display_errors', 1);
ini_set('display_startup_errors', 1);
error_reporting(E_ALL);

session_start();
session_destroy();

header("Location: index.php");
exit();
```

Côté client :

gestion_panier.php :

Ce fichier est appelé lorsque le client clique sur "Ajouter au panier" depuis le catalogue. Il s'agit du moteur de la fonction d'ajout. Si l'id du produit reçu par la méthode POST existe déjà dans le tableau du panier en session, alors on met à jour la quantité en ajoutant la

nouvelle quantité à celle déjà stockée dans la session. Sinon, cette nouvelle quantité est stockée directement dans la session.

```php
// On vérifie à l'intérieur du panier si le produit avec cet id a déjà été ajouté ou pas
if (isset($_SESSION["panier"][$id_produit])) {

    // Si oui, on rajoute à la quantité existante la nouvelle quantité
    $_SESSION["panier"][$id_produit] = $_SESSION["panier"][$id_produit] + $quantite;

    // Sinon, on assigne cette nouvelle quantité directement
} else {
    $_SESSION["panier"][$id_produit] = $quantite;
}
```

modifier_quantite.php :

Ce fichier est le moteur des petits boutons "+" et "-" de la page panier. Il récupère l'id du produit et l'action souhaitée directement via l'URL en utilisant la méthode GET.

```
localhost:8080/exercices/boulangerie/modifier_quantite.php?id_produit=3&action=ajouter
```

```
localhost:8080/exercices/boulangerie/modifier_quantite.php?id_produit=3&action=retirer
```

Le fichier vérifie d'abord si le produit existe bien dans la superglobale $_SESSION. Ensuite, selon l'action (soit "ajouter" ou "retirer"), il incrémente ou décrémente simplement la quantité.

```php
if ($action == "ajouter") {
    // S'il y a déjà une quantité, on ajoute de 1
    $_SESSION["panier"][$id_produit]++;
} elseif ($action == "retirer") {
    // S'il y a déjà une quantité, on diminue de 1
    $_SESSION["panier"][$id_produit]--;
```

Si la quantité tombe à zéro après une soustraction, on utilise la fonction unset() pour détruire la ligne et retirer définitivement le produit du panier.

```php
// S'il n'y a aucune quantité, on supprime le produit du panier
if ($_SESSION["panier"][$id_produit] == 0) {
    unset($_SESSION["panier"][$id_produit]);
}
```

Enfin, le fichier termine son exécution en renvoyant instantanément le client vers la page précédente grâce à la fonction header(), ce qui rend le rafraîchissement transparent pour l'utilisateur, comme s'il n'avait jamais changé de page.

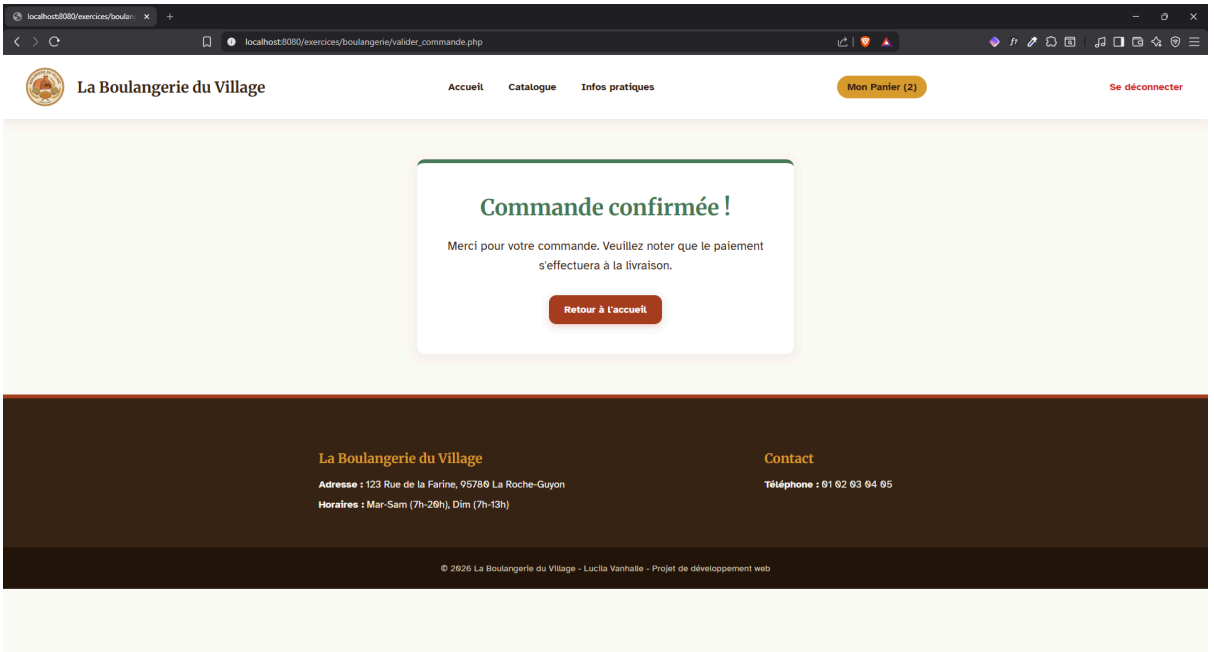
supp_panier.php :

Ce fichier est également appelé par la méthode GET depuis la page du panier. Il permet de supprimer un produit du panier en utilisant la fonction unset().

```php
<?php
ini_set('display_errors', 1);
ini_set('display_startup_errors', 1);
error_reporting(E_ALL);

session_start();

if (isset($_GET['id_produit'])) {
    $id_produit = $_GET['id_produit'];
    unset($_SESSION["panier"][$id_produit]);
    header("Location: panier.php");
    exit();
}
```

valider_commande.php :

Ce fichier est à la fois le moteur de validation mais affiche également le message de confirmation.



Il permet de créer une commande lorsque le client clique sur “Valider la commande” depuis le panier.

Il donne un numéro de commande aléatoire grâce à la fonction rand().

```
// On donne un numéro de commande aléatoire, le 10000 c'est pour garantir les 5 chiffres
$num_commande = "CMD-" . rand(10000, 99999);
```

Il bascule le numéro de commande généré, la date du jour (la date de commande) ainsi que le statut “En préparation” dans la table commande grâce à une requête préparée.

```
// CURRENT_DATE() pour mettre la date du jour au moment de valider
$stmt = $conn->prepare("INSERT INTO commande (num_commande, date_commande, statut, id_utilisateur) VALUES (?, CURRENT_DATE(), 'En préparation', ?)");
$stmt->bind_param("si", $num_commande, $_SESSION["utilisateur_id"]);
```

Une fois la commande créée, le serveur a besoin de savoir quel numéro d'id vient de lui être attribué pour lier les produits et la commande dans la table intermédiaire details_commande. Pour cela, on utilise la fonction $conn->insert_id.

```
    // On récupère l'id de la commande qui vient juste d'être généré
    $last_id = $conn->insert_id;
```

La commande et les détails de commande sont en base de données donc le panier en mémoire ne sert plus à rien. C’est pour cela qu’on le détruit avec la fonction unset().

```
    // On vide le panier après la validation
    unset($_SESSION["panier"]);
```

Côté admin :

Dans tous ces fichiers admin, on inclut une vérification du role pour garantir l’accès à ces fonctionnalités seulement à l’admin.

modifier_statut.php :

Ce fichier reçoit par la méthode GET l'id de la commande et le nouveau statut choisi par l’admin (soit "Prête à être livrée" ou "Livrée"). Selon le nouveau statut, on fait une requête préparée avec l’instruction UPDATE.

```
    // On va update le statut selon la variable new_statut envoyée
    if ($new_statut == "prete") {
        $stmt = $conn->prepare("UPDATE commande SET statut = 'Prête à être livrée' WHERE num_commande = ?");
        $stmt->bind_param('s', $num_commande);
        $stmt->execute();
        $stmt->close();
    } elseif ($new_statut == "livree") {
        $stmt = $conn->prepare("UPDATE commande SET statut = 'Livrée' WHERE num_commande = ?");
        $stmt->bind_param('s', $num_commande);
        $stmt->execute();
        $stmt->close();
    }
```

supp_produit.php :

Ce fichier reçoit (toujours pas la méthode GET) l'id d'un produit du catalogue que l'admin souhaite retirer. On utilise une requête préparée avec l'instruction DELETE FROM.

```
13        $stmt = $conn->prepare("DELETE FROM produit WHERE id=?");
14        $stmt->bind_param("i", $id_produit);
```

update_produit.php :

Ce fichier est le moteur du formulaire de modification d'un produit du catalogue. Il reçoit toutes les nouvelles données (nom, prix, description) de manière invisible avec la méthode POST. On utilise une requête préparée avec l'instruction UPDATE.

```
19        $stmt = $conn->prepare("UPDATE produit SET nom=?, description=?, prix=?, id_categorie=? WHERE id=?");
20        $stmt->bind_param("ssdii", $nom, $description, $prix, $categorie, $id_produit);
```

1.3. Sécurité du site

a) Hachage et authentification

Si la base de données est compromise, le hacker ne doit pas pouvoir lire les mots de passe. J'ai donc transformé le mot de passe en une chaîne de caractères illisible lors de l'inscription avec la fonction password_hash(). Lors de la connexion, j'utilise la fonction password_verify() pour comparer le mot de passe tapé en clair par l'utilisateur avec la version brouillée stockée en base.

b) Sécurisation de la base de données

Il ne faut jamais faire confiance aux saisies des utilisateurs car on risque les injections SQL, c'est-à-dire qu'un client peut taper du code SQL malveillant dans un formulaire pour supprimer ou modifier les tables de la base de données. Pour contrer cela, je sépare strictement l'instruction SQL des données saisies par l'utilisateur grâce aux requêtes préparées. Ces dernières utilisent la méthode prepare() où l'on inclut l'instruction SQL en mettant un ? à chaque valeur provenant d'une saisie utilisateur. Ensuite, je lie mes variables avec la méthode

bind_param() en indiquant le type de données que va recevoir la base de données ainsi que la variable contenant la valeur reçue.

c) Sanitisation côté affichage

La sanitisation du côté affichage sert à contrer les failles XSS. Si une donnée stockée en base de données contient du code JavaScript malveillant (par exemple, dans le nom d'un produit ou d'un utilisateur) et qu'on l'affiche tel quel, le navigateur du client va l'exécuter. Il faut donc nettoyer cette donnée avant de l'afficher pour transformer le code en simple texte inoffensif. Pour cela, j'ai utilisé la fonction htmlspecialchars().

d) Vérification de la méthode de réception des données

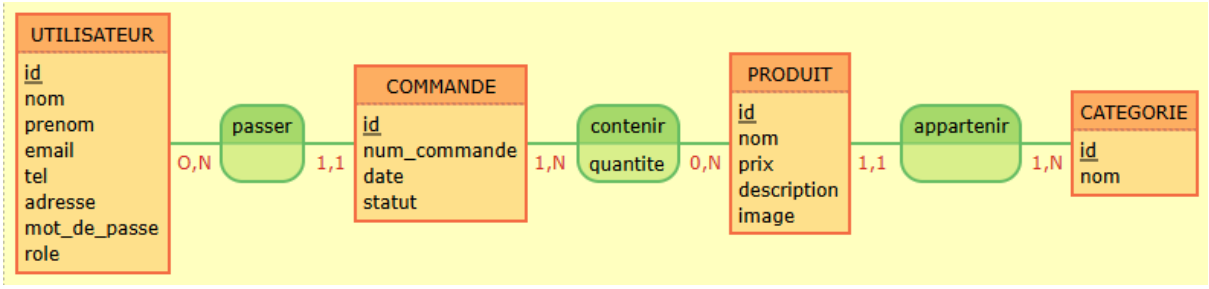
Les informations sensibles comme un formulaire d'inscription ou l'ajout d'un produit doivent voyager de manière invisible avec la méthode POST.
À l'inverse, les petites actions transparentes comme retirer un produit du panier peuvent passer par l'URL avec la méthode GET.

e) Sécuriser l'accès des fonctionnalités admin

Un client curieux pourrait deviner et taper l'URL admin.php ou ajouter_produit.php directement dans son navigateur. Il faut donc vérifier avec une condition si l'utilisateur qui tente d'entrer a le bon role, si ce n'est pas le cas, on la renvoie à l'accueil.

2. Modélisation de la base de données

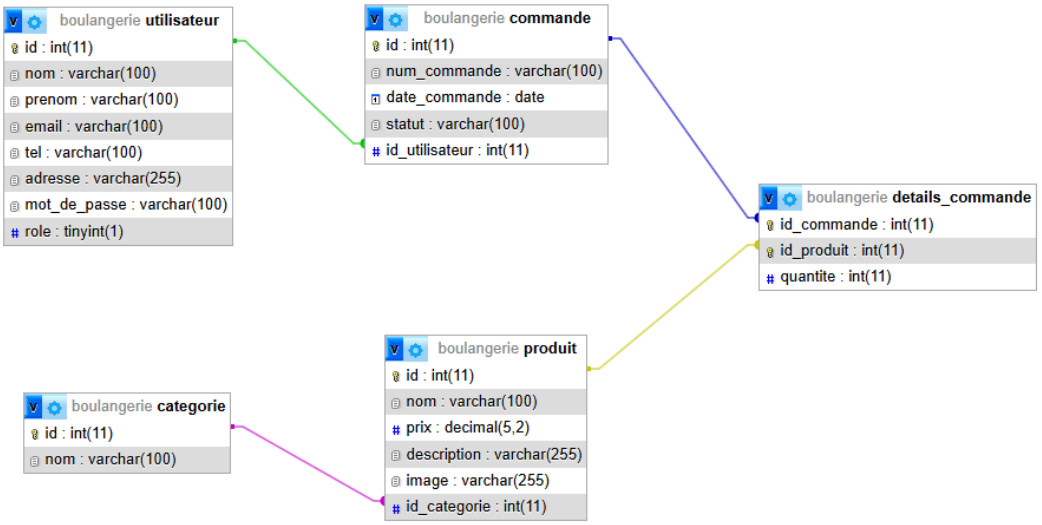
2.1. Modèle Conceptuel des Données (MCD)



2.2. Modèle Logique de Données (MLD)

utilisateur (id, nom, prenom, email, telephone, adresse, mot_de_passe)
commande (id, num_commande, date, statut, #id_utilisateur)
details_commande (#id_commande, #id_produit, quantite)
produit (id, nom, prix, description, image, #id_categorie)
categorie (id, nom)

2.3. Présentation des tables



2.4. Étapes de la création de la base de données MySQL

```sql
CREATE DATABASE boulangerie;
```

```sql
CREATE TABLE utilisateur (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nom VARCHAR(100) NOT NULL,
    prenom VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL,
    tel VARCHAR(100),
    adresse VARCHAR(255),
    mot_de_passe VARCHAR(100) NOT NULL,
    role BOOL NOT NULL
);
```

```sql
CREATE TABLE commande (
    id INT PRIMARY KEY AUTO_INCREMENT,
    num_commande VARCHAR(100) NOT NULL,
    date_commande DATE NOT NULL,
    statut VARCHAR(100) NOT NULL,
    id_utilisateur INT NOT NULL,
    FOREIGN KEY (id_utilisateur) REFERENCES utilisateur(id)
);
```

```sql
CREATE TABLE categorie (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nom VARCHAR(100) NOT NULL
    );
```

```sql
CREATE TABLE produit (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nom VARCHAR(100) NOT NULL,
    prix DECIMAL(5,2) NOT NULL,
    description VARCHAR(255),
    image VARCHAR(255),
    id_categorie INT NOT NULL,
    FOREIGN KEY(id_categorie) REFERENCES categorie(id)
);
```

```sql
CREATE TABLE details_commande (
    id_commande INT NOT NULL,
    id_produit INT NOT NULL,
    quantite INT NOT NULL,
    PRIMARY KEY(id_commande, id_produit),
    FOREIGN KEY (id_commande) REFERENCES commande(id),
    FOREIGN KEY (id_produit) REFERENCES produit(id)
);
```

Avant de faire une remarque sur la limite de caractères à 255, sachez que j'en ai parlé dans mon rapport et que j'ai modifié mes limites (voir p.4 du rapport).

J'ai choisi d'inclure un attribut role de type BOOL dans la table utilisateur pour pouvoir distinguer les clients et l'administrateur.

J'ai choisi DECIMAL(5,2) pour le prix du produit car cela permet d'éviter des erreurs d'arrondis et les prix peuvent aller jusqu'à 999,99€ sans problème (même si c'est un peu excessif pour une boulangerie).

J'ai choisi de transformer l'association contenir en table portant le nom details_commande pour plus de clarté dans la base de données. Celle-ci gère la relation entre les commandes et les produits tout en incluant la quantité.

Définir l'admin

J'ai d'abord créé un compte avec la fonctionnalité d'inscription. Comme il s'est mis en role 0 (donc client) par défaut, j'ai ensuite fait une requête SQL directement dans la base de données pour mettre à jour le role à 1.

```sql
UPDATE utilisateur
SET role = 1
WHERE email = 'admin@boulangerie.com';
```

Insertion des catégories

```sql
INSERT INTO categorie (nom)
VALUES
('pains'),
('viennoiseries'),
('patisseries');
```

Insertion des produits

```sql
INSERT INTO produit(nom, prix, description, image, id_categorie)
VALUES
('Baguette', '1.10','La star de nos fournées, à la croûte dorée et la mie alvéolée. Incontournable à chaque heure de la journée.', 'img/baguette.png', 1),
('Ficelle', '0.80','Pour les petits creux ou l\'apéritif, notre baguette fine et ultra-croustillante.', 'img/ficelle.png', 1),
('Bâtard', '1.20','Le pain de nos grands-pères. Une forme rustique et une mie dense, parfaite pour accompagner un bon plat en sauce ou un plateau de charcuterie.', 'img/batard.png', 1),
('Boulot', '1.50','Le classique des déjeuners de famille. Un pain généreux et tout rond, dont on aime rompre la croûte farinée à la main.', 'img/boulot.png', 1),
('Pain de campagne', '2.50','Un pain de caractère au levain naturel, avec une croûte épaisse qui garde toute sa fraîcheur.', 'img/pain_campagne.png', 1),
('Pain de mie', '2.00','La douceur absolue de la boulangerie. Une mie briochée, moelleuse à souhait, qui fera le bonheur de vos toasts et de vos enfants.', 'img/pain_mie.png', 1),
('Pain viennois', '1.30','Entre pain et brioche, sa texture fine et sa croûte dorée en font le compagnon idéal des goûters gourmands avec une barre de chocolat.', 'img/pain_viennois.png', 1),
('Pain complet', '2.20','La force du grain entier. Un pain authentique et rustique, riche en fibres, pour une dégustation saine qui a du goût.', 'img/pain_complet.png', 1),
('Pain aux céréales', '2.40','Un mélange savoureux de graines craquantes pour un pain riche en goût et en énergie.', 'img/pain_cereales.png', 1),
('Pain de seigle', '2.30','Le compagnon idéal de vos plateaux de fromages et de vos dégustations marines.', 'img/pain_seigle.png', 1);
```

```sql
INSERT INTO produit(nom, prix, description, image, id_categorie)
VALUES
('Croissant', '1.20', 'Le classique de notre artisan : un feuilletage pur beurre, croustillant à souhait et fondant à l\'intérieur.', 'img/croissant.png', 2),
('Pain au chocolat', '1.30', 'L\'incontournable du goûter, préparé avec notre pâte feuilletée levée et deux barres de chocolat noir intense.', 'img/pain_choco.png', 2),
('Pain suisse', '1.50', 'Une gourmandise généreuse alliant la douceur d\'une crème pâtissière maison et le croquant des pépites de chocolat.', 'img/pain_suisse.png', 2),
('Croissant aux amandes', '1.60', 'Une recette de tradition nappée d\'une crème d\'amandes onctueuse et parsemée d\'amandes effilées grillées.', 'img/croissant_amandes.png', 2),
('Chausson aux pommes', '1.40', 'Un feuilletage doré renfermant une compote de pommes fondante, doucement mijotée avec une pointe de cannelle.', 'img/chausson_pommes.png', 2),
('Pain aux raisins', '1.50', 'Aussi appelé Escargot, cette spirale gourmande est garnie d\'une crème vanillée et de raisins secs soigneusement sélectionnés.', 'img/pain_raisins.png', 2),
('Torsade', '1.50', 'Un ruban de pâte feuilletée pur beurre, tressé avec une crème pâtissière onctueuse et des pépites de chocolat noir.', 'img/torsade.png', 2),
('Chouquettes', '2.00', 'Douze petites bouchées de pâte à chou légères et aériennes, généreusement saupoudrées de gros grains de sucre.', 'img/chouquettes.png', 2),
('Brioche tressée', '4.50', 'La véritable brioche de notre enfance : une tresse dorée à la main, au bon goût de beurre frais et à la mie incroyablement aérée.', 'img/brioche_tressee.png', 2),
('Pain Briaché à Chocolat', '2.80', 'Une création gourmande à la mie ultra moelleuse, généreusement truffée de pépites de chocolat noir, au lait et blanc.', 'img/brioche_3_choco.png', 2);
```

```sql
INSERT INTO produit(nom, prix, description, image, id_categorie)
VALUES
('Saint-honoré', 3.50, 'Le roi des pâtisseries : un socle de pâte feuilletée craquant, des petits choux caramélisés et une crème onctueuse d\'une légèreté absolue.', 'img/saint_honore.png', 3),
('Opéra', 4.00, 'Une partition gourmande où s\'équilibrent la force du café et la douceur d\'une ganache au chocolat noir intense.', 'img/opera.png', 3),
('Mille-feuille', 3.50, 'Le chef-d\'œuvre de notre pâtissier : trois couches de feuilletage doré qui craquent sous la dent pour laisser place à une crème vanillée onctueuse.', 'img/mille_feuille.png', 3),
('Religieuse', 3.80, 'Un duo de choux généreux reliés par une dentelle de crème au beurre, pour les amoureux de tradition pâtissière.', 'img/religieuse.png', 3),
('Tarte Bourdaloue', 3.50, 'Le charme des poires pochées fondantes sur un lit de crème d\'amandes, le tout niché dans une pâte sablée pur beurre.', 'img/tarte_bourdaloue.png', 3),
('Financier', 2.00, 'Un petit lingot de gourmandise au beurre noisette et à l\'amande, parfait pour accompagner votre café au Village.', 'img/financier.png', 3),
('Macarons', 1.50, 'Un écrin de douceur avec une coque croquante et un cœur fondant, décliné selon les saveurs de saison.', 'img/macarons.png', 3),
('Paris-Brest', 4.50, 'Une couronne de pâte à choux parsemée d\'amandes, renfermant un trésor de crème mousseline au praliné maison.', 'img/paris_brest.png', 3),
('Moka', 3.80, 'L\'authenticité d\'une génoise aérienne mariée à une crème au beurre onctueuse, délicatement parfumée au café fraîchement moulu.', 'img/moka.png', 3);
```

3. Guide d'Installation

Pour faire tourner le projet de la Boulangerie du Village en local, voici les étapes à suivre. C'est assez rapide si vous avez déjà un environnement de développement prêt.

3.1. Pré-requis (Installation d'un serveur local)

Avant tout, il faut un serveur local pour interpréter le PHP et gérer la base de données MySQL.

- ➔ **Le plus simple :** Utiliser Docker, il suffit d'utiliser une image php:apache et un container mysql.
- ➔ **Navigateur :** N'importe lequel (Chrome, Firefox, etc).

3.2. Importation du dump de la base de données (.sql)

Il faut d'abord importer les tables pour que le site puisse afficher les produits et gérer les comptes.

1. Allez sur votre phpMyAdmin (localhost:8081).
2. Allez dans l'onglet "Importer", choisissez mon fichier boulangerie.sql qui est à la racine du dossier.
3. Validez. Normalement, toutes les tables (utilisateur, produit, commande, details_commande, categorie) se créent toutes seules avec quelques données de test que j'ai déjà injectées.

3.3. Configuration de la connexion à la base de données

C'est l'étape la plus importante pour que le PHP puisse se connecter à la base de données.

1. Ouvrez le fichier dbconfig.php avec votre éditeur de code (VS Code ou autre).
2. Vérifiez les infos de connexion :
    - Les identifiants dans le fichier dbconfig.php (le serveur, l'utilisateur et le mot de passe) doivent correspondre exactement aux variables d'environnement définies dans le fichier docker-compose.yml.

3.4. Déploiement et lancement du site

Dernière étape pour voir le résultat :

1. Copiez mon dossier de projet dans votre répertoire de serveur (par exemple dans le dossier www sur Docker).
2. Avant d'aller sur le navigateur, il faut ouvrir un terminal dans le dossier du projet et exécuter la commande docker-compose up -d pour démarrer les services.
3. Tapez localhost:8080/boulangerie/index.php dans votre barre d'adresse.

Et voilà ! Vous devriez arriver sur la page d'accueil. Pour tester la partie admin, vous pouvez utiliser ces identifiants :

email : admin@boulangerie.com

mot de passe : admin